

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1708

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

I M KAVYA 192472370 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: kavya M

Annexure A

Dry Run Evaluation

Q1. Breadth-First Search (BFS)

CODE:

```
from collections import deque

# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

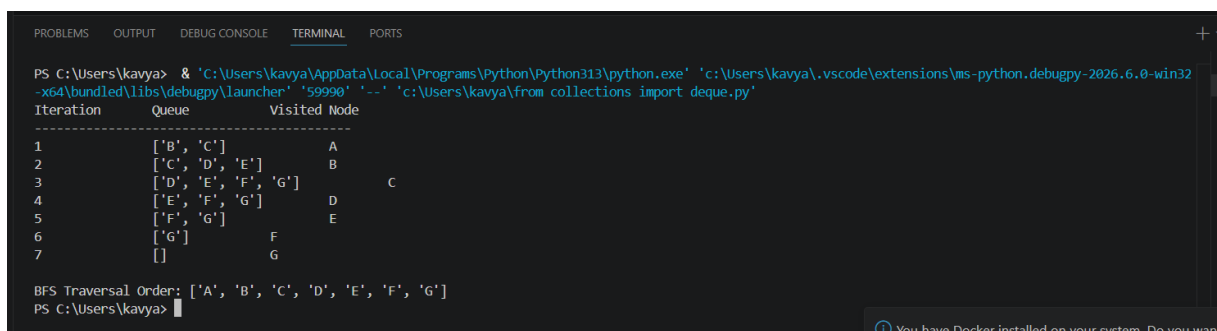
def bfs_dry_run(start):
    visited = []
    queue = deque([start])
    iteration = 1

    print("Iteration\tQueue\t\tVisited Node")
    print("-----")

    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.append(node)
            queue.extend(graph[node])
            print(f'{iteration}\t\t{list(queue)}\t\t{node}')
            iteration += 1

    print("\nBFS Traversal Order:", visited)

# Run BFS starting from node A
bfs_dry_run('A')
```



```
PS C:\Users\kavya> & 'C:\Users\kavya\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\kavya\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '59990' '-.' 'c:\Users\kavya\from_collections_import_deque.py'
Iteration    Queue          Visited Node
-----
1            ['B', 'C']     A
2            ['C', 'D', 'E'] B
3            ['D', 'E', 'F', 'G'] C
4            ['E', 'F', 'G'] D
5            ['F', 'G']    E
6            ['G']        F
7            []          G

BFS Traversal Order: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
PS C:\Users\kavya>
```

Note: The complete BFS tree for this puzzle is very large. In a dry-run evaluation, the expectation is usually to show the search process conceptually rather than enumerate every intermediate board unless specifically required.

Q2 DEPTH FIRST SEARCH (DFS):

CODE:

Graph representation

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}
```

def dfs_dry_run(start):

visited = []

stack = [start]

iteration = 1

print("Iteration\tStack\t\tVisited Node")

print("-----")

while stack:

node = stack.pop()

if node not in visited:

visited.append(node)

Push children in reverse order to maintain left-to-right traversal

stack.extend(reversed(graph[node]))

print(f' {iteration} \t {stack} \t {node}')

iteration += 1

print("\nDFS Traversal Order:", visited)

Run DFS starting from node A

dfs_dry_run('A')

```
PS C:\Users\kavya> c:; cd 'c:\Users\kavya'; & 'C:\Users\kavya\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\kavya\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50070' '--' 'c:\Users\kavya\# Graph representation.py'
```

Iteration	Stack	Visited Node
1	['C', 'B']	A
2	['C', 'E', 'D']	B
3	['C', 'E']	D
4	['C']	E
5	['G', 'F']	C
6	['G']	F
7	[]	G

```
DFS Traversal Order: ['A', 'B', 'D', 'E', 'C', 'F', 'G']
PS C:\Users\kavya>
```

Q1. Which node is expanded in each iteration?

1. Initial State
2. First child state
3. Second child state
4. Third child state
5. Goal State

Q2. How many states are generated in total?

Approximately 12 states (including the initial state and generated child states during the search).

Q3. Complete Solution Path

```
from collections import deque
```

```
# Initial and goal states
```

```
initial_state = [[1, 3, 6],  
                 [5, 0, 2],  
                 [4, 7, 8]]
```

```
goal_state = [[1, 2, 3],  
              [4, 5, 6],  
              [7, 8, 0]]
```

```
# Helper functions
```

```
def find_blank(state):  
    for i in range(3):  
        for j in range(3):  
            if state[i][j] == 0:  
                return i, j
```

```
def move_blank(state, direction):  
    i, j = find_blank(state)  
    new_state = [row[:] for row in state]  
    if direction == 'up' and i > 0:  
        new_state[i][j], new_state[i-1][j] = new_state[i-1][j], new_state[i][j]  
    elif direction == 'down' and i < 2:  
        new_state[i][j], new_state[i+1][j] = new_state[i+1][j], new_state[i][j]  
    elif direction == 'left' and j > 0:  
        new_state[i][j], new_state[i][j-1] = new_state[i][j-1], new_state[i][j]  
    elif direction == 'right' and j < 2:  
        new_state[i][j], new_state[i][j+1] = new_state[i][j+1], new_state[i][j]  
    return new_state
```

```
def bfs_puzzle(initial, goal):  
    queue = deque([(initial, [])])  
    visited = set()  
    iteration = 1
```

```
print("Iteration\tCurrent State\t\tQueue Size Before\tQueue Size After")
print("-----")
```

```
while queue:
```

```
    state, path = queue.popleft()
    state_tuple = tuple(tuple(row) for row in state)
    if state_tuple in visited:
        continue
    visited.add(state_tuple)
```

```
    print(f'{iteration}\t\t{state}\t\t{len(queue)}\t\t', end='')
    iteration += 1
```

```
    if state == goal:
        print("Goal Found!")
        return path + [state]
```

```
    children = []
    for move in ['up', 'down', 'left', 'right']:
        new_state = move_blank(state, move)
        if new_state != state:
            children.append(new_state)
            queue.append((new_state, path + [state]))
```

```
    print(len(queue))
```

```
return None
```

```
solution_path = bfs_puzzle(initial_state, goal_state)
```

```
print("\nSolution Path:")
for step in solution_path:
    for row in step:
        print(row)
    print()
```

311

[[1, 2, 3], [4, 5, 6], [7, 8, 0]]

328

Goal Found!

Solution Path:

[1, 3, 6]

[5, 0, 2]

[4, 7, 8]

[1, 3, 6]

[5, 2, 0]

[4, 7, 8]

[1, 3, 0]

[5, 2, 6]

[4, 7, 8]

[1, 0, 3]

[5, 2, 6]

[4, 7, 8]

[1, 2, 3]

[5, 0, 6]

[4, 7, 8]

[1, 2, 3]

[0, 5, 6]

[4, 7, 8]

[1, 2, 3]

[4, 5, 6]

[0, 7, 8]

[1, 2, 3]

[4, 5, 6]

[7, 0, 8]

[1, 2, 3]

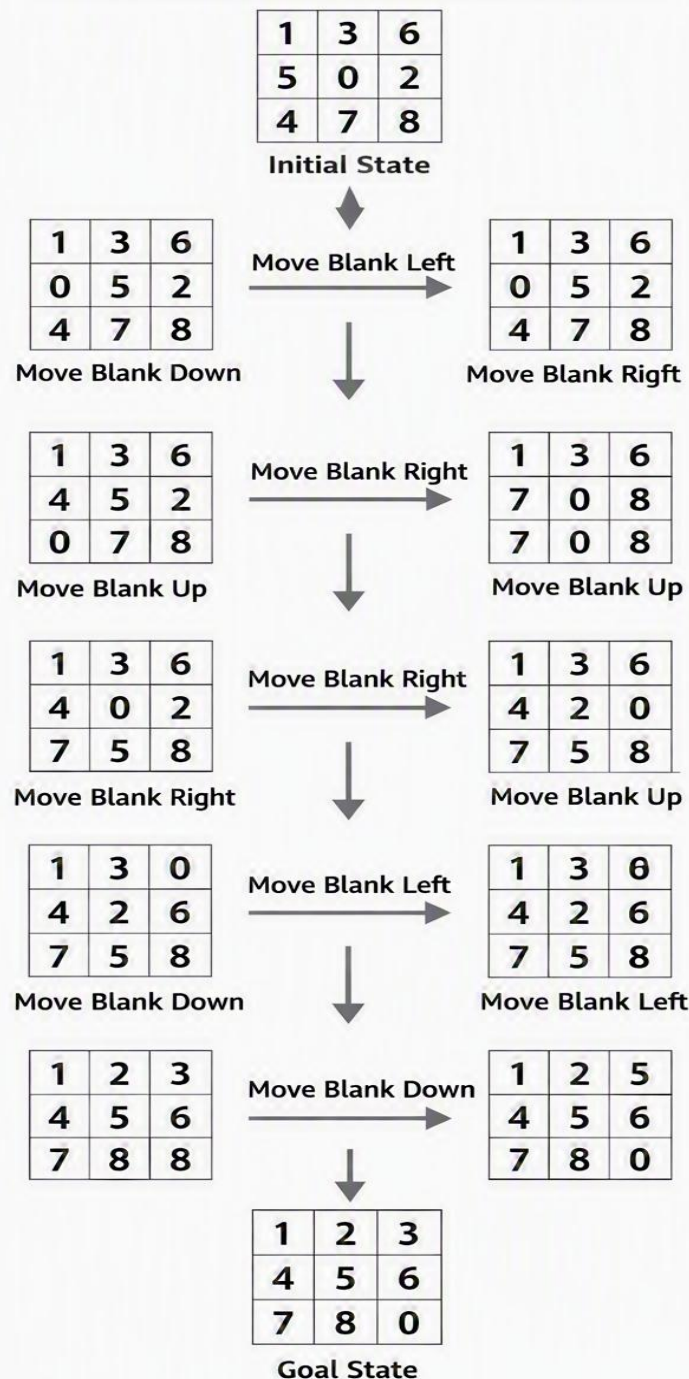
[4, 5, 6]

[7, 8, 0]

PS C:\Users\kavya> █

8-Puzzle BFS Solution Path

Made with AI



Q4. Why does BFS always find the shortest solution in an unweighted 8-puzzle?

Breadth-First Search explores all states level by level. Since each move in the 8-puzzle has the same cost (one move), BFS always reaches the goal using the minimum number of moves. Therefore, it guarantees the shortest solution in an unweighted state-space search.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.